

Investigación Operativa

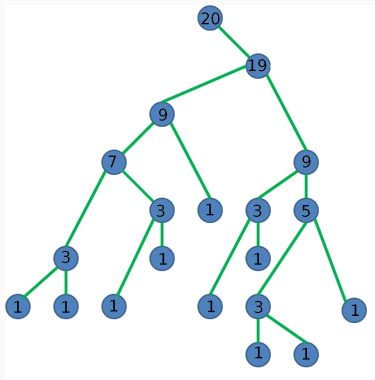
Árboles

Facundo Gutiérrez - Nazareno Faillace Mullen

Departamento de Matemática - FCEN - UBA

Repaso de definiciones

- Llamamos *árbol* a un grafo conexo y acíclico.
- Llamamos *bosque* a un grafo acíclico (unión disjunta de árboles).
- Llamamos *hoja* a un vértice de grado 1.



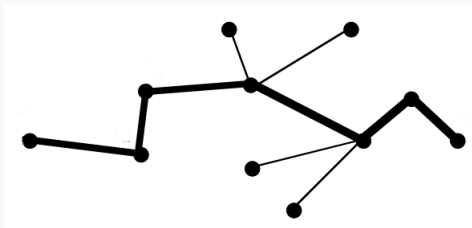
Ejercicio 1

Proposición: Si todo vértice de un grafo $\mathcal{G}$ tiene grado al menos 2, entonces $\mathcal{G}$ tiene un ciclo.

Ejercicio 1

Proposición: Si todo vértice de un grafo $\mathcal{G}$ tiene grado al menos 2, entonces $\mathcal{G}$ tiene un ciclo.

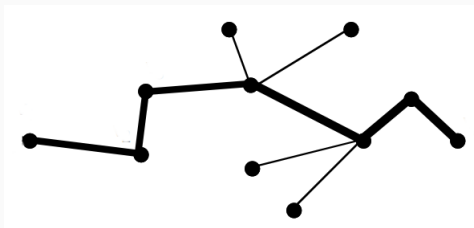
Demostración: Supongamos que $\mathcal{G}$ no tiene ciclos. Consideremos P algún camino simple maximal. Sea v algún extremo de P . Como $d(v) \geq 2 \dots$



Ejercicio 1

Proposición: Si todo vértice de un grafo $\mathcal{G}$ tiene grado al menos 2, entonces $\mathcal{G}$ tiene un ciclo.

Demostración: Supongamos que $\mathcal{G}$ no tiene ciclos. Consideremos P algún camino simple maximal. Sea v algún extremo de P . Como $d(v) \geq 2 \dots$



... existe un vecino de v distinto de su vecino inmediato en P . Llamémosle w . Si $w \in P$, entonces $\mathcal{G}$ tendría un ciclo. Si $w \notin P$, entonces podemos extender a P , contradiciendo su condición de maximal. Concluimos que $\mathcal{G}$ necesariamente tiene un ciclo.

Ejercicio 2

Proposición: La cantidad de hojas de un árbol es mayor o igual al grado máximo.

Ejercicio 2

Proposición: La cantidad de hojas de un árbol es mayor o igual al grado máximo.

Demostración: Sea v un nodo de grado máximo. De cada arista incidente en v consideremos un camino maximal con extremo en v . Todos estos caminos son disjuntos (exceptuando el extremo v), ¿por qué?

Ejercicio 2

Proposición: La cantidad de hojas de un árbol es mayor o igual al grado máximo.

Demostración: Sea v un nodo de grado máximo. De cada arista incidente en v consideremos un camino maximal con extremo en v . Todos estos caminos son disjuntos (exceptuando el extremo v), ¿por qué? Porque si no fueran disjuntos, cerrarían un ciclo.

Razonando análogamente al ejercicio anterior, podemos ver que cada extremo de estos caminos debe ser una hoja, de donde concluimos el resultado del enunciado.

Caracterización de un árbol

Son equivalentes:

- $\mathcal{G}$ es un árbol (conexo y acíclico)
- Dados dos vértices en $\mathcal{G}$ existe un único camino que los une.
- $\mathcal{G} = (\mathbb{V}, \mathbb{E})$ es conexo y $|\mathbb{E}| = |\mathbb{V}| - 1$
- $\mathcal{G} = (\mathbb{V}, \mathbb{E})$ es acíclico y $|\mathbb{E}| = |\mathbb{V}| - 1$

Caracterización de un árbol

Son equivalentes:

- $\mathcal{G}$ es un árbol (conexo y acíclico)
- Dados dos vértices en $\mathcal{G}$ existe un único camino que los une.
- $\mathcal{G} = (\mathbb{V}, \mathbb{E})$ es conexo y $|\mathbb{E}| = |\mathbb{V}| - 1$
- $\mathcal{G} = (\mathbb{V}, \mathbb{E})$ es acíclico y $|\mathbb{E}| = |\mathbb{V}| - 1$

¿Cuántas aristas tiene como mínimo un grafo con c componentes conexas?

Caracterización de un árbol

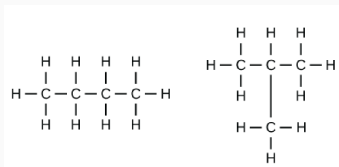
Son equivalentes:

- $\mathcal{G}$ es un árbol (conexo y acíclico)
- Dados dos vértices en $\mathcal{G}$ existe un único camino que los une.
- $\mathcal{G} = (\mathbb{V}, \mathbb{E})$ es conexo y $|\mathbb{E}| = |\mathbb{V}| - 1$
- $\mathcal{G} = (\mathbb{V}, \mathbb{E})$ es acíclico y $|\mathbb{E}| = |\mathbb{V}| - 1$

¿Cuántas aristas tiene como mínimo un grafo con c componentes conexas? $|\mathbb{V}| - c$

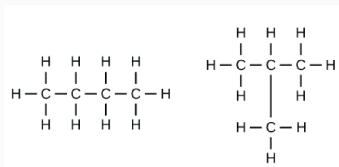
Ejercicio 3

Un hidrocarburo saturado es una molécula C_pH_q en la cual cada átomo de carbono tiene 4 enlaces y cada átomo de hidrógeno tiene exactamente un enlace. Ninguna sucesión de enlaces forma un ciclo. Mostrar que para todo $p \in \mathbb{Z}_{>0}$ la única forma de que un hidrocarburo saturado C_pH_q exista es si $q = 2p + 2$.



Ejercicio 3

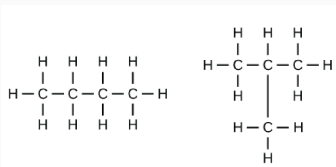
Un hidrocarburo saturado es una molécula C_pH_q en la cual cada átomo de carbono tiene 4 enlaces y cada átomo de hidrógeno tiene exactamente un enlace. Ninguna sucesión de enlaces forma un ciclo. Mostrar que para todo $p \in \mathbb{Z}_{>0}$ la única forma de que un hidrocarburo saturado C_pH_q exista es si $q = 2p + 2$.



Solución: La cantidad de vértices es $p + q$. Por ser una molécula el grafo es conexo (si no es conexo serían 2 o más moléculas). Como no se forman ciclos, entonces el grafo con el que se modela la molécula es un árbol y por lo tanto tiene $p + q - 1$ aristas.

Ejercicio 3

Un hidrocarburo saturado es una molécula C_pH_q en la cual cada átomo de carbono tiene 4 enlaces y cada átomo de hidrógeno tiene exactamente un enlace. Ninguna sucesión de enlaces forma un ciclo. Mostrar que para todo $p \in \mathbb{Z}_{>0}$ la única forma de que un hidrocarburo saturado C_pH_q exista es si $q = 2p + 2$.



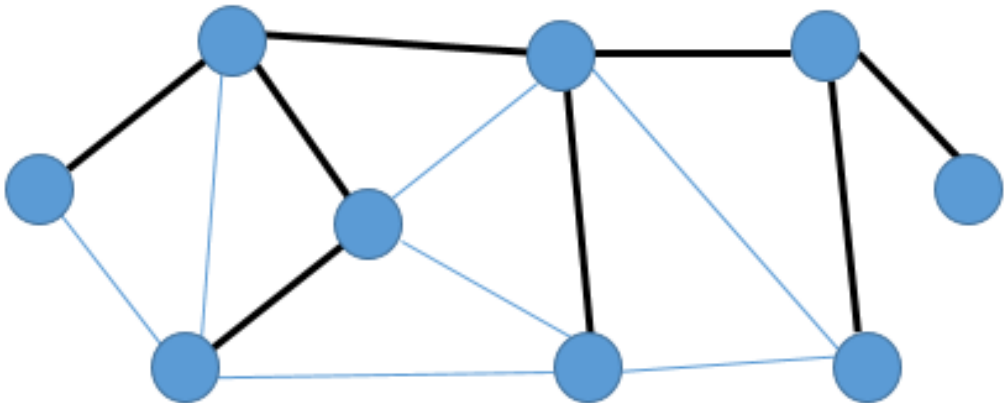
Solución: La cantidad de vértices es $p + q$. Por ser una molécula el grafo es conexo (si no es conexo serían 2 o más moléculas). Como no se forman ciclos, entonces el grafo con el que se modela la molécula es un árbol y por lo tanto tiene $p + q - 1$ aristas.

Usando la fórmula de la suma de los grados, concluimos que:

$$4p + q = 2(p + q - 1) \iff q = 2p + 2$$

Árbol generador

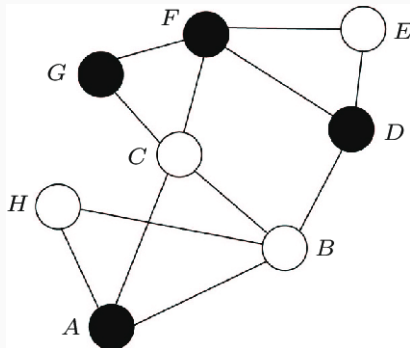
- Repaso: sea $\mathcal{G} = (V_G, E_G)$, $\mathcal{H} = (V_H, E_H)$ es un subgrafo de $\mathcal{G}$ si $V_H \subseteq V_G$ y $E_H \subseteq E_G$.
- Repaso: $\mathcal{H}$ es subgrafo generador de $\mathcal{G}$ si $V_H = V_G$.
- Un árbol generador de $\mathcal{G}$ es un subgrafo generador de $\mathcal{G}$ que es un árbol.



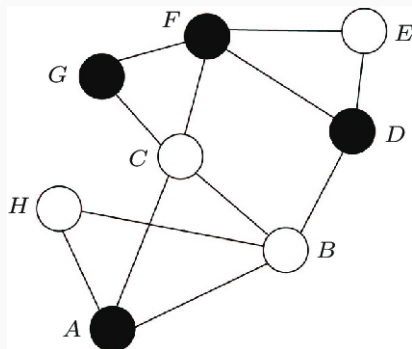
Ejercicio 4

Se tienen N ciudades y M rutas que las conectan. En el plano original es posible llegar de cualquier ciudad a otra mediante combinaciones de rutas.

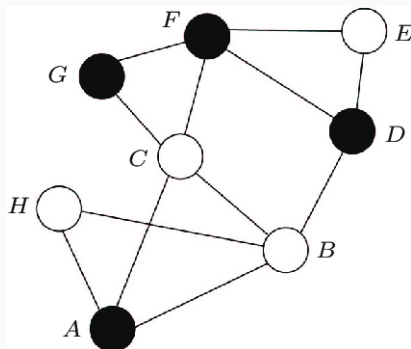
Las ciudades se dividen en dos grupos. Hay A del grupo 1 y B del grupo 2 (o sea que $A + B = N$). Se quiere renovar el pavimento de algunas rutas de forma tal que de todas las ciudades de grupo 1 se llegue a alguna ciudad de grupo 2 usando solamente rutas con pavimento renovado. ¿Cuál es la menor cantidad de rutas que hacen falta pavimentar?



Ejercicio 4

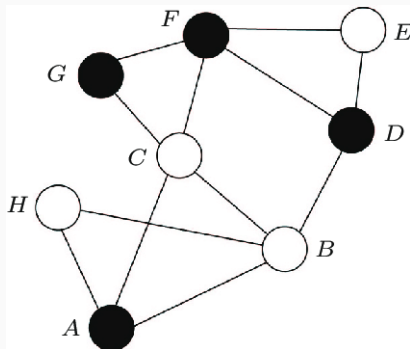


Ejercicio 4



Solución: De cada ciudad del grupo 1, necesariamente vamos a tener que pavimentar alguna ruta que se conecte con ella (en caso contrario, esa ciudad jamás podrá llegar a una del grupo 2). Por lo tanto, debemos pavimentar al menos A rutas. ¿Podemos lograr nuestro objetivo utilizando solo A rutas?

Ejercicio 4



Solución: De cada ciudad del grupo 1, necesariamente vamos a tener que pavimentar alguna ruta que se conecte con ella (en caso contrario, esa ciudad jamás podrá llegar a una del grupo 2). Por lo tanto, debemos pavimentar al menos A rutas. ¿Podemos lograr nuestro objetivo utilizando solo A rutas?

¿Hace falta que el grafo original sea conexo? ¿Cómo podemos verlo usando lo que vimos hasta ahora?

Repaso de definiciones

- Un grafo $\mathcal{G} = (\mathbb{V}, \mathbb{E}, w)$ se dice *pesado* si sus aristas tienen asociadas un cierto número, dado por $w : \mathbb{E} \rightarrow \mathbb{R}$

Repaso de definiciones

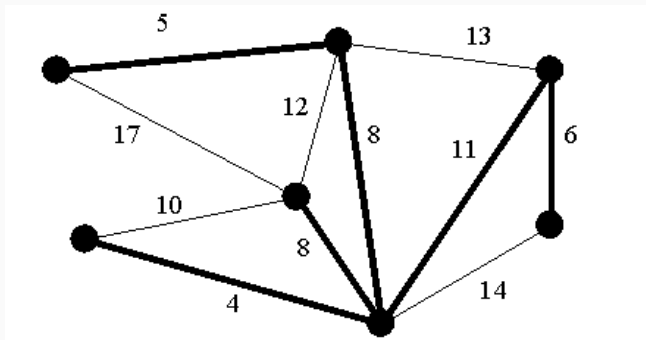
- Un grafo $\mathcal{G} = (\mathbb{V}, \mathbb{E}, w)$ se dice *pesado* si sus aristas tienen asociadas un cierto número, dado por $w : \mathbb{E} \rightarrow \mathbb{R}$
- $\mathcal{T}$ es un árbol *generador mínimo* de $\mathcal{G}$, si es un subgrafo de $\mathcal{G}$ que es árbol (conexo y acíclico), generador ($\mathbb{V}_{\mathcal{T}} = \mathbb{V}$) y minimiza $\sum_{e \in \mathbb{E}_{\mathcal{T}}} w(e)$

Repaso de definiciones

- Un grafo $\mathcal{G} = (\mathbb{V}, \mathbb{E}, w)$ se dice *pesado* si sus aristas tienen asociadas un cierto número, dado por $w : \mathbb{E} \rightarrow \mathbb{R}$
- $\mathcal{T}$ es un árbol *generador mínimo* de $\mathcal{G}$, si es un subgrafo de $\mathcal{G}$ que es árbol (conexo y acíclico), generador ($\mathbb{V}_{\mathcal{T}} = \mathbb{V}$) y minimiza $\sum_{e \in \mathbb{E}_{\mathcal{T}}} w(e)$
- El *cuello de botella* de un árbol se define como $\max_{e \in \mathbb{E}_{\mathcal{T}}} w(e)$

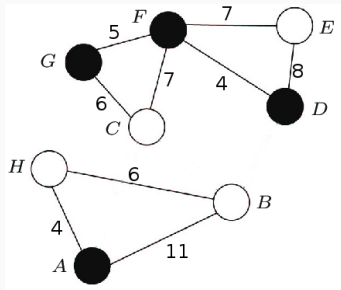
Repaso de definiciones

- Un grafo $\mathcal{G} = (\mathbb{V}, \mathbb{E}, w)$ se dice *pesado* si sus aristas tienen asociadas un cierto número, dado por $w : \mathbb{E} \rightarrow \mathbb{R}$
- $\mathcal{T}$ es un árbol *generador mínimo* de $\mathcal{G}$, si es un subgrafo de $\mathcal{G}$ que es árbol (conexo y acíclico), generador ($\mathbb{V}_{\mathcal{T}} = \mathbb{V}$) y minimiza $\sum_{e \in \mathbb{E}_{\mathcal{T}}} w(e)$
- El *cuello de botella* de un árbol se define como $\max_{e \in \mathbb{E}_{\mathcal{T}}} w(e)$



Ejercicio 5

Se tienen N ciudades y M rutas que conectan un par de ciudades. Las ciudades se dividen en dos grupos. Hay A del grupo 1 y B del grupo 2 (o sea que $A + B = N$). Se quiere renovar el pavimento de algunas rutas de forma tal que de todas las ciudades de grupo 1 se llegue a alguna ciudad de grupo 2 usando solamente rutas con pavimento renovado. Cada ruta e tiene un costo w_e . ¿Cuál es el menor costo con el que se puede lograr el objetivo?



- $\mathcal{T}$ es un árbol *enraizado* si distinguimos a un nodo de $\mathcal{T}$ al cual llamamos la *raíz*.

Árbol enraizado

- $\mathcal{T}$ es un árbol *enraizado* si distinguimos a un nodo de $\mathcal{T}$ al cual llamamos la *raíz*.
- Puede ser dirigido o no. En caso de ser dirigido, hay dos formas naturales en las que se dirigen las aristas que son acercándose a la raíz o alejándose de la raíz.

Árbol enraizado

- $\mathcal{T}$ es un árbol *enraizado* si distinguimos a un nodo de $\mathcal{T}$ al cual llamamos la *raíz*.
- Puede ser dirigido o no. En caso de ser dirigido, hay dos formas naturales en las que se dirigen las aristas que son acercándose a la raíz o alejándose de la raíz.
- Definimos la *profundidad* de un nodo como la distancia a la raíz.

Árbol enraizado

- $\mathcal{T}$ es un árbol *enraizado* si distinguimos a un nodo de $\mathcal{T}$ al cual llamamos la *raíz*.
- Puede ser dirigido o no. En caso de ser dirigido, hay dos formas naturales en las que se dirigen las aristas que son acercándose a la raíz o alejándose de la raíz.
- Definimos la *profundidad* de un nodo como la distancia a la raíz.
- La *altura* de un árbol enraizado está dada por la máxima profundidad en el árbol.

Árbol enraizado

- $\mathcal{T}$ es un árbol *enraizado* si distinguimos a un nodo de $\mathcal{T}$ al cual llamamos la *raíz*.
- Puede ser dirigido o no. En caso de ser dirigido, hay dos formas naturales en las que se dirigen las aristas que son acercándose a la raíz o alejándose de la raíz.
- Definimos la *profundidad* de un nodo como la distancia a la raíz.
- La *altura* de un árbol enraizado está dada por la máxima profundidad en el árbol.
- Si un vértice v es un predecesor en un camino desde la raíz a otro vértice w , decimos que v es el *padre* de w (y que w es un hijo de v). A dos vértices que tienen un mismo padre se los llaman *hermanos*.

Árbol enraizado

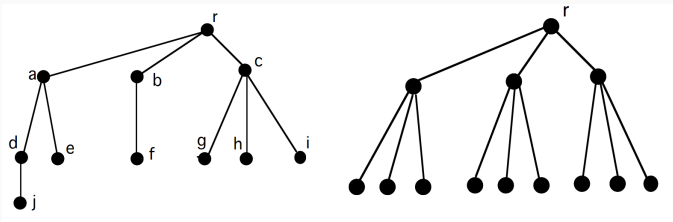
- $\mathcal{T}$ es un árbol *enraizado* si distinguimos a un nodo de $\mathcal{T}$ al cual llamamos la *raíz*.
- Puede ser dirigido o no. En caso de ser dirigido, hay dos formas naturales en las que se dirigen las aristas que son acercándose a la raíz o alejándose de la raíz.
- Definimos la *profundidad* de un nodo como la distancia a la raíz.
- La *altura* de un árbol enraizado está dada por la máxima profundidad en el árbol.
- Si un vértice v es un predecesor en un camino desde la raíz a otro vértice w , decimos que v es el *padre* de w (y que w es un hijo de v). A dos vértices que tienen un mismo padre se los llaman *hermanos*.
- Un vértice v es un *ancestro* de w si v se encuentra en el camino que une a la raíz con w . Además en este caso, se dice que w es un descendiente de v .

Árbol enraizado

- Un vértice que no tiene hijos se lo llama una *hoja*. Todo vértice que tenga al menos un hijo se lo llama *interno*.

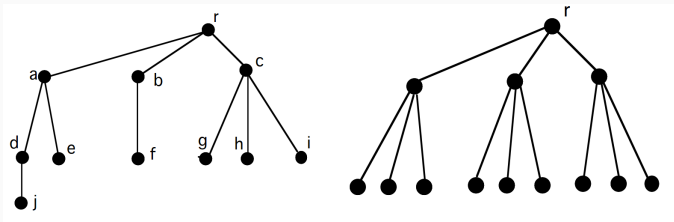
Árbol enraizado

- Un vértice que no tiene hijos se lo llama una *hoja*. Todo vértice que tenga al menos un hijo se lo llama *interno*.
- Un árbol enraizado se dice *k-ario*, si todo vértice tiene a lo sumo k hijos. Si todas las hojas tienen la misma altura, y todo vértice que no es hoja tiene exactamente k hijos, se dice que es un *árbol k-ario completo*



Árbol enraizado

- Un vértice que no tiene hijos se lo llama una *hoja*. Todo vértice que tenga al menos un hijo se lo llama *interno*.
- Un árbol enraizado se dice *k-ario*, si todo vértice tiene a lo sumo k hijos. Si todas las hojas tienen la misma altura, y todo vértice que no es hoja tiene exactamente k hijos, se dice que es un *árbol k-ario completo*



Recordar: la altura de un árbol k -ario completo con n vértices es $O(\log_k((k-1)m))$

Ejercicio: Si H e I denotan la cantidad de hojas y de nodos internos respectivamente, probar que en un árbol k -ario completo, vale que

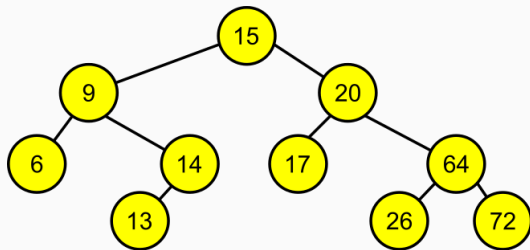
$$H = (k - 1)I + 1$$

Árbol binario de búsqueda

Un *árbol binario de búsqueda* guarda en cada nodo un valor. Es un árbol 2-ario que cumple que el subárbol izquierdo de cualquier nodo (de existir) contiene valores menores que el del nodo en cuestión, y el subárbol derecho (de existir) valores mayores.

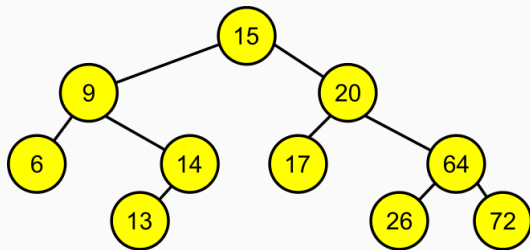
Árbol binario de búsqueda

Un *árbol binario de búsqueda* guarda en cada nodo un valor. Es un árbol 2-ario que cumple que el subárbol izquierdo de cualquier nodo (de existir) contiene valores menores que el del nodo en cuestión, y el subárbol derecho (de existir) valores mayores.



Árbol binario de búsqueda

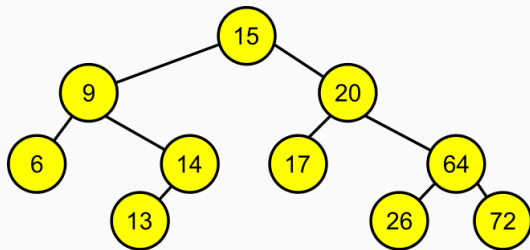
Un *árbol binario de búsqueda* guarda en cada nodo un valor. Es un árbol 2-ario que cumple que el subárbol izquierdo de cualquier nodo (de existir) contiene valores menores que el del nodo en cuestión, y el subárbol derecho (de existir) valores mayores.



- *pre-order* (raíz, izq, der):

Árbol binario de búsqueda

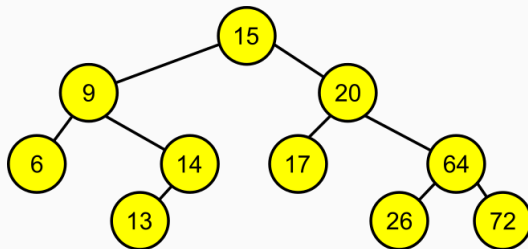
Un *árbol binario de búsqueda* guarda en cada nodo un valor. Es un árbol 2-ario que cumple que el subárbol izquierdo de cualquier nodo (de existir) contiene valores menores que el del nodo en cuestión, y el subárbol derecho (de existir) valores mayores.



- *pre-order* (raíz, izq, der): 15, 9, 6, 14, 13, 20, 17, 64, 26, 72

Árbol binario de búsqueda

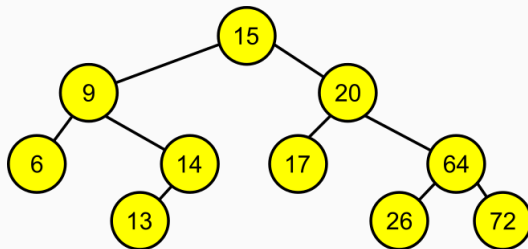
Un *árbol binario de búsqueda* guarda en cada nodo un valor. Es un árbol 2-ario que cumple que el subárbol izquierdo de cualquier nodo (de existir) contiene valores menores que el del nodo en cuestión, y el subárbol derecho (de existir) valores mayores.



- *pre-order* (raíz, izq, der): 15, 9, 6, 14, 13, 20, 17, 64, 26, 72
- *in-order* (izq, raíz, der):

Árbol binario de búsqueda

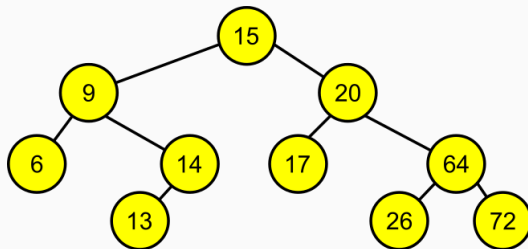
Un *árbol binario de búsqueda* guarda en cada nodo un valor. Es un árbol 2-ario que cumple que el subárbol izquierdo de cualquier nodo (de existir) contiene valores menores que el del nodo en cuestión, y el subárbol derecho (de existir) valores mayores.



- *pre-order* (raíz, izq, der): 15, 9, 6, 14, 13, 20, 17, 64, 26, 72
- *in-order* (izq, raíz, der): 6, 9, 13, 14, 15, 17, 20, 26, 64, 72

Árbol binario de búsqueda

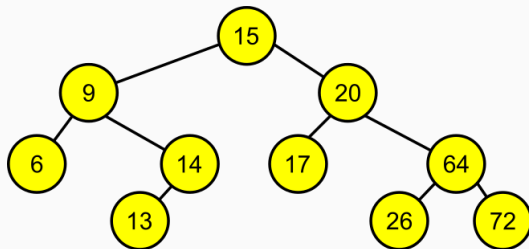
Un *árbol binario de búsqueda* guarda en cada nodo un valor. Es un árbol 2-ario que cumple que el subárbol izquierdo de cualquier nodo (de existir) contiene valores menores que el del nodo en cuestión, y el subárbol derecho (de existir) valores mayores.



- *pre-order* (raíz, izq, der): 15, 9, 6, 14, 13, 20, 17, 64, 26, 72
- *in-order* (izq, raíz, der): 6, 9, 13, 14, 15, 17, 20, 26, 64, 72
- *post-order* (izq, der, raíz):

Árbol binario de búsqueda

Un *árbol binario de búsqueda* guarda en cada nodo un valor. Es un árbol 2-ario que cumple que el subárbol izquierdo de cualquier nodo (de existir) contiene valores menores que el del nodo en cuestión, y el subárbol derecho (de existir) valores mayores.



- *pre-order* (raíz, izq, der): 15, 9, 6, 14, 13, 20, 17, 64, 26, 72
- *in-order* (izq, raíz, der): 6, 9, 13, 14, 15, 17, 20, 26, 64, 72
- *post-order* (izq, der, raíz): 6, 13, 14, 9, 17, 26, 72, 64, 20, 15

Heap

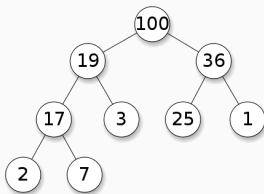
Al igual que el árbol binario de búsqueda, un *heap* es un árbol binario donde vamos a guardar valores en los nodos, pero el árbol binario en cuestión es *casi* completo (con excepción del último nivel, que se llena de izquierda a derecha).

En un *max-heap* el valor de cada nodo es mayor al valor de cada uno de sus hijos. Dado un heap, podemos *insertar* un nuevo valor, *borrar* el mayor elemento (la raíz), y *consultar* el valor en la raíz.

Heap

Al igual que el árbol binario de búsqueda, un *heap* es un árbol binario donde vamos a guardar valores en los nodos, pero el árbol binario en cuestión es *casi* completo (con excepción del último nivel, que se llena de izquierda a derecha).

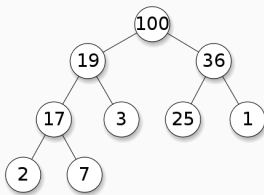
En un *max-heap* el valor de cada nodo es mayor al valor de cada uno de sus hijos. Dado un heap, podemos *insertar* un nuevo valor, *borrar* el mayor elemento (la raíz), y *consultar* el valor en la raíz.



Heap

Al igual que el árbol binario de búsqueda, un *heap* es un árbol binario donde vamos a guardar valores en los nodos, pero el árbol binario en cuestión es *casi* completo (con excepción del último nivel, que se llena de izquierda a derecha).

En un *max-heap* el valor de cada nodo es mayor al valor de cada uno de sus hijos. Dado un heap, podemos *insertar* un nuevo valor, *borrar* el mayor elemento (la raíz), y *consultar* el valor en la raíz.



¿Cómo implementaríamos cada una de ellas? ¿Qué complejidades temporales tienen?